

# Lab overview

The lab covers the concepts of configuring a web server behind an Elastic Load Balancer (ELB). The lab presents a scenario where users are not able to reach a website hosted on an Amazon Elastic Compute Cloud (Amazon EC2) instance behind a load balancer.

Your goal is to troubleshoot and fix all the misconfigured issues to allow accessing the website page using the load balancer DNS name while ensuring the solution can still work if one of the web servers instances is down.

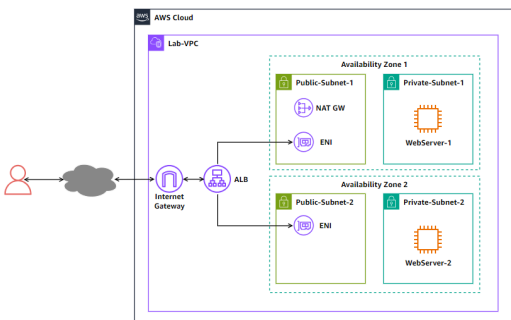
## Objectives

By the end of this lab, I should be able to do the following:

- Examine the current configuration of the Application Load Balancer (ALB) and the website running on EC2 instances behind the ALB.
- Troubleshoot and identify the issues in the existing configuration which are preventing users from accessing the website.
- Remediate the configuration to allow users to access the website behind the ALB and ensure the solution is highly available.

## Lab environment

The following diagram shows the lab scenario:

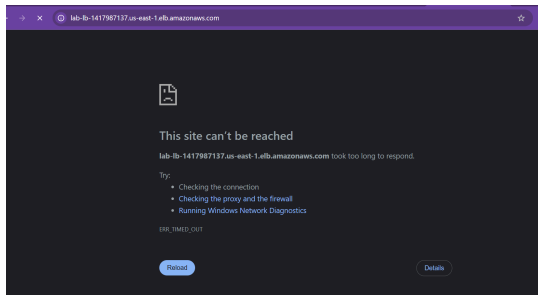


*Image description: The following list details the major resources in the lab:*

- A VPC hosting all the lab resources.
- The VPC has two public subnets and two private subnets across two availability zones.
- The VPC has an internet gateway allowing public access from the internet.
- A NAT gateway in one of the public subnets to provide outbound access from the private subnets.
- An Application Load Balancer (ALB) with an elastic network interface (ENI) in each of the public subnets.
- An EC2 instance in each of the private subnets to host the website.

# Task 1: Testing the web server reachability from the internet

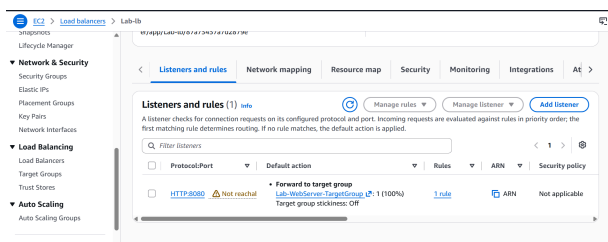
In this task, I'll test the web server reachability from the internet to verify the issue. Since the web server is in a private subnet behind the load balancer, I'll attempt reaching the web server using the load balancer DNS name



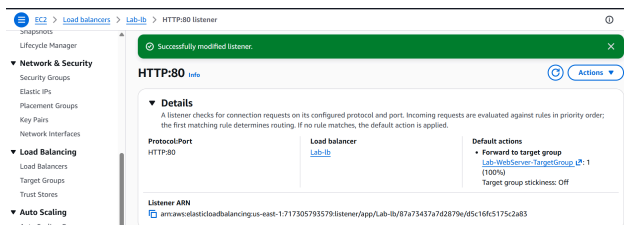
**Task complete:** I tested the website reachability from the internet and verified that it cannot be reached.

# Task 2: Troubleshooting and remediating website reachability issues

In this task, I'll identify the issues in the existing configuration and remediate the configuration so the website can be accessed from the internet using the load balancer DNS name.

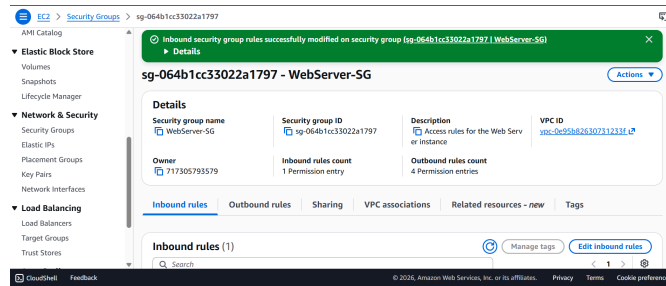


I observed the listener under the *Protocol:Port* column. I noticed that the listener is configured as HTTP:8080 and it is *Not reachable*. I recalled that the HTTP service on the web servers is running on port 80. This is clearly misconfigured on the load balancer listener.



I successfully remediated the load balancer listener.

I noticed that the security group attached to the second web server instance is the same as the first instance of which there are no ingress rules configured. So the inbound rules are missing and must be remediated. For security best practice, the ingress rule should allow HTTP traffic on TCP port 80 from the load balancer security group *ALB-SG*.



To verify that I resolved all the issues, I can simply attempt to access the website from my browser using the load balancer DNS name.



Instance ID: i-0588e1631d5ecfb6e

**task complete:** I identified the issues in the existing configuration and remediated the configuration so the website can be accessed from the internet using the load balancer DNS name

## Task 3: Ensuring the solution is highly available

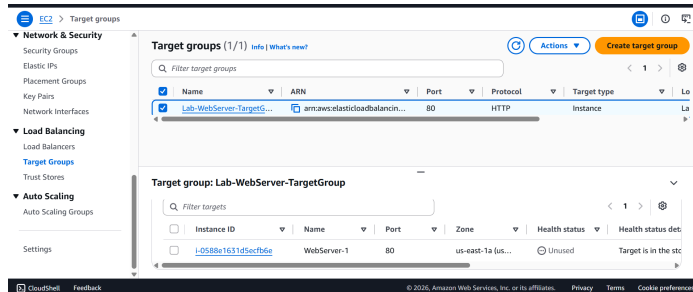
After remediating the website reachability issues, I need to improve the solution resiliency to mitigate against any potential failure in the instance hosting the website.

In this task, I'll configure my solution to be highly available to ensure that the website is still reachable even if one instance is down. First, I'll stop the instance currently hosting the website and check if the website is still reachable or not.

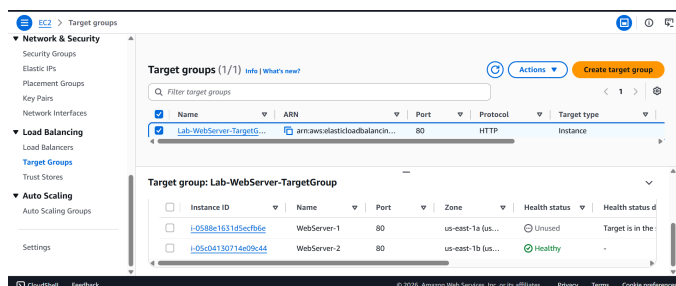


My browser page fails to connect to the web server. Even though I have a second instance *WebServer-2* hosting the website, the website is not reachable when the first instance is down.

When configuring a load balancer, one needs to configure a target group. The target group is a logical container of all the targets (instances in this scenario) that the load balancer can send traffic to. I'll check the target group configuration under the load balancer.



I noticed that the only instance in this target group is *WebServer-1*. This instance initially has a health status of *Unhealthy* and then *Unused* because it was stopped to simulate instance failure. The second instance *WebServer-2* was not added to the group in the initial configuration. Now I'll remediate these two issues.

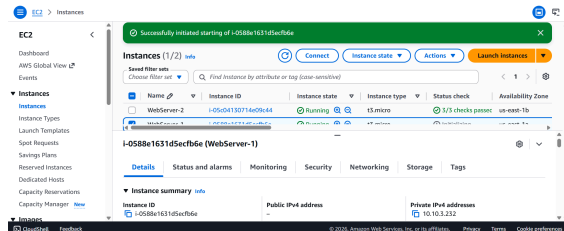


To verify my solution, I'll attempt to access the website using the load balancer DNS name while the *WebServer-1* instance is still down (stopped).

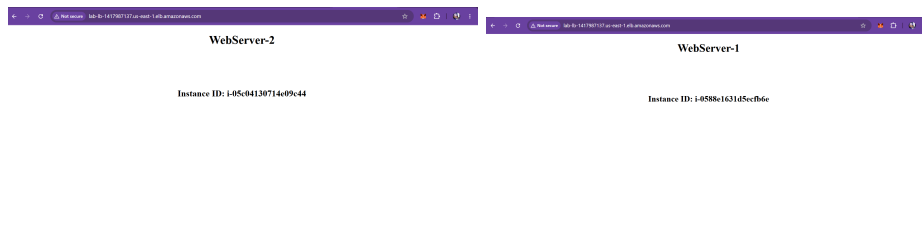


The browser returns the website page which includes the web server instance name *WebServer-2* where the website is hosted. The page also displays the EC2 instance ID for *WebServer-2*.

Now, I'll start the *WebServer-1* instance to verify that load balancing is working between the two instances.



I returned to the browser tab where I'll access the load balancer DNS name and refreshed the browser multiple times. I noticed that the page alternates between the two instances as indicated by the web server name and instance id displayed on each page.



**Task complete:** I configured my solution to be highly available to ensure that the website is still reachable even if one instance is down.

## Conclusion

I have successfully done the following:

- Examined the current configuration of the Application Load Balancer and website running on EC2 instances behind the ALB.
- Troubleshooted and identified the issues in the existing configuration which are preventing users from accessing the website.
- Remediated the configuration to allow users to access the website behind the ALB and ensured the solution is highly available.

